

PREGUNTAS SOBRE MUTATION TESTING

31 ENERO 2025

NOMBRE DEL ESTUDIANTE: Mario Daniel Sajvin Gómez

1. *¿Qué es Mutation Testing y cuál es su propósito principal?*
Es una técnica de prueba de software que evalúa la calidad de un conjunto de casos de prueba al introducir modificaciones deliberadas (mutaciones) en el código fuente. Su propósito principal es medir qué tan bien los casos de prueba pueden detectar errores, asegurando que los tests cubran adecuadamente el comportamiento esperado del sistema.
2. *¿Qué significa que un mutante haya “sobrevivido” en una prueba de mutación?*
Un mutante "sobrevive" cuando los casos de prueba no logran fallar al ejecutarse sobre el código modificado. Esto indica que el conjunto de pruebas no es lo suficientemente robusto como para detectar esa mutación, lo cual puede sugerir una falta de cobertura o pruebas poco efectivas.
3. *¿Qué significa que in mutante haya “muerto” en una prueba de mutación?*
Un mutante "muere" cuando los casos de prueba fallan al ejecutarse sobre el código modificado. Esto demuestra que el conjunto de pruebas es efectivo para identificar errores introducidos deliberadamente, validando su capacidad de encontrar defectos.
4. *¿Cómo se generan los mutantes en Mutation Testing? Menciona al menos tres tipos de modificaciones que se pueden hacer en el código.*
Los mutantes se generan al modificar pequeñas partes del código fuente de manera controlada. Algunos ejemplos de modificaciones son:
 - Operadores aritméticos: cambiar + por - o * por /.
 - Operadores relacionales: cambiar > por < o == por !=.
 - Eliminación de condiciones: remover partes de una expresión condicional (por ejemplo, cambiar if (a && b) a if (a)).
5. *¿Qué son los mutantes equivalentes y por qué representan un desafío en Mutation Testing?*
Los mutantes equivalentes son aquellos que, a pesar de las modificaciones, tienen el mismo comportamiento que el código original y no pueden ser detectados por los casos de prueba. Representan un desafío porque generan falsos negativos en los resultados de las pruebas y requieren una validación manual para identificarlos, lo que puede ser costoso y consume tiempo.

6. Menciona al menos dos herramientas de Mutation Testing y los lenguajes de programación con los que son compatibles.

- Pitest: compatible con Java. Es una de las herramientas más populares para pruebas de mutación en este lenguaje.
- Stryker: compatible con JavaScript, TypeScript, y C#. Es conocida por su integración con frameworks modernos y facilidad de uso.

Aplicando:

7. Suponga usted que tiene el siguiente código en Java:

java

```
public int suma(int a, int b) {  
    return a + b;  
}
```

Si aplicamos Mutation Testing, mencione una posible mutación que se podría generar y cómo debería detectarla un buen caso de prueba.

Mutación posible:

```
public int suma(int a, int b) {  
    return a - b;  
}
```

Un buen caso de prueba sería:

```
@Test  
public void testSuma() {  
    assertEquals(5, suma(2, 3)); // Prueba que 2 + 3 sea 5  
}
```

8. Si después de ejecutar Mutation Testing en un proyecto, el 80% de los mutantes sobreviven, ¿qué podría usted concluir sobre la calidad de las pruebas unitarias existentes?

La supervivencia del 80% de los mutantes sugiere que las pruebas unitarias existentes no son efectivas para cubrir completamente el comportamiento del sistema ni para detectar posibles errores. Esto puede significar:

- Falta de cobertura: muchas partes del código no están siendo ejecutadas por los casos de prueba.
- Casos de prueba débiles: las pruebas no validan correctamente los resultados esperados o no están diseñadas para capturar errores específicos.

Se recomienda analizar los resultados y crear nuevos casos de prueba para mejorar la cobertura y la capacidad de detección de errores.

9. *Explique cómo el Mutation Testing puede ayudar a mejorar la calidad del código y la cobertura de pruebas en un proyecto de software.*

Mutation Testing permite identificar áreas del código que no están bien cubiertas por las pruebas, destacando casos donde los errores podrían pasar inadvertidos. Al aplicar esta técnica:

- Fortalece los casos de prueba: obliga a diseñar pruebas más específicas y robustas.
- Aumenta la confianza en las pruebas: asegura que el conjunto de pruebas puede detectar errores en cambios futuros.
- Mejora la cobertura de pruebas: identifica partes del código que necesitan ser validadas con más rigor.

En general, ayuda a construir un código más confiable y reduce el riesgo de errores en producción.

10. *En un sistema con una gran cantidad de código, ¿qué estrategias se pueden usar para mitigar el alto costo computacional del Mutation Testing?*

Para reducir el costo computacional del Mutation Testing en sistemas grandes, se pueden emplear las siguientes estrategias:

1. Ejecución en áreas críticas: aplicar Mutation Testing solo en módulos críticos o en partes del código con alto riesgo de errores.
2. Ejecución incremental: realizar pruebas de mutación solo en los cambios recientes o en el código modificado, en lugar de todo el sistema.
3. Filtrado de mutantes: configurar la herramienta para generar únicamente mutantes relevantes, evitando aquellos que son redundantes o triviales.
4. Ejecución en paralelo: utilizar múltiples hilos o máquinas para distribuir la carga de trabajo.
5. Pruebas en horarios no laborales: programar las pruebas de mutación para que se ejecuten durante la noche o en momentos de baja actividad del equipo de desarrollo.
6. Priorización de mutantes: ejecutar primero los mutantes que tienen mayor probabilidad de sobrevivir, optimizando el análisis.

Estas estrategias ayudan a balancear el costo y el beneficio del Mutation Testing en proyectos de gran escala.